EcoExplain-VLM — Code, Research Plan & Implementation Audit

Prepared from the repository contents, generated artifacts, and validated benchmark outputs. Focus: evidence-based project archaeology, not narrative assumptions.

Verified evidence used: dataset row count, class distributions, zero-shot benchmark ranking, fine-tuning result, and code-path tracing through scripts, notebooks, and source modules.

Executive summary

• The repository is materially real and operational: it contains the full geospatial export pipeline, DIP/Otsu processing, JSONL dataset construction, stratified split creation, multiple VLM evaluation paths, and a Qwen fine-tuning workflow.

• The codebase is stronger than a purely speculative prototype, but the project's research narrative is not fully matched by the underlying evidence: the documentation and slides are more polished than the raw methodology and validation story in several places.

• The strongest evidence is the concrete artifact set: 366 processed evidence records, stratified train/dev/test splits, four zero-shot benchmark models, and a Qwen LoRA run with 67.3% accuracy on the test split.

• The main methodological caveat is that the project mixes auto-generated candidate labels (from spectral change detection) with human reference labels, and the code explicitly uses a policy that filters/keeps boxes based on reference_class when present. This is a useful engineering pipeline, but it is not a pure ground-truth benchmark by itself.

Project status at a glance

Area	Status	Evidence	Assessment
Earth Engine export pipeline	Implemented	scripts/01_export_gee.py; src/gee_pipeline.py	Operational and connected to real ROI/date config.
DIP / Otsu disturbance extraction	Implemented	src/dip_utils.py	Core logic matches the motion from NDVI/NBR masks to triplets and s
Dataset assembly	Implemented	scripts/03_build_dataset.py; notebooks/02_VLM_json_dataset.ipynb	366 JSONL records present with multimodal metadata.
Freeze + splits	Implemented	scripts/05_freeze_jsonl_and_splits.py	Stratified split writing and a manifest are present.
Zero-shot VLM evaluation	Implemented	src/vlm_eval.py; results/vlm_combo_comparison.csv	Four models were benchmarked; Qwen2-VL was strongest at 5
Fine-tuning	Implemented	src/vlm_finetune.py; scripts/run_finetune_qwen.py	Qwen LoRA fine-tune executed and evaluated at 67.3% on test.
Research-plan fidelity	Mixed	README + slides + code	The polished plan is broadly consistent with actual code but not always
Explainability claim strength	Partial	src/text_utils.py; evidence cards	Text explanations are present but best understood as narrative summa

Verified repository facts

• Dataset row count from the actual JSONL artifact: 366 rows.

• Model benchmark result at zero-shot test time: Qwen2-VL best accuracy = 56.4% (n=55 test examples).

• Fine-tuned Qwen result: 67.3% accuracy on the same 55-example test split.

• The project stores candidate classes and reference classes separately: candidate_class is generated automatically, while reference_class is human-annotated when available and is intentionally used by the code policy for box filtering.

• The repo distinguishes extensive evidence cards from older pilot cards; freeze/split logic writes the final dataset and train/dev/test patch lists under data/splits/.

Planned vs implemented

Research plan element	Planned or claimed	Actual implementation status	Evidence
GEE multi-source export	Core project pillar	Implemented	src/gee_utils.py and scripts/01_export_gee.py
Condition C image-to-label benchmark	Central objective	Implemented	candidate_class in dataset.jsonl and evaluation scripts
DIP-based disturbance detection	Yes	Implemented	src/dip_utils.py; build_disturbance_triplets()
Human validation / reference labels	Yes, as quality control	Partially implemented	reference_class exists but only for a subset; apply_reference_class_policy()
Zero-shot VLM benchmarking	Yes	Implemented	results/vlm_combo_comparison.json
Fine-tuning pipeline	Yes	Implemented	scripts/run_finetune_qwen.py and src/vlm_finetune.py
Explainability / XAI validation	Strongly framed	Partial/engineering-level	src/text_utils.py and evidence narrative generation, not full causal explanatio
Large-scale full study area rollout	Claimed in roadmap	Partially present as a pilot + scale configs/scripts	configs, scripts, and data folder structure indicate a complete prototype more

Key algorithmic and design findings

• The class labels are strongly linked to spectral evidence. In src/dip_utils.py, apply_reference_class_policy() explicitly says Otsu detects spectral drops only; stable/increase patches should not carry many disturbance boxes and decline patches keep the largest boxes.

• The code is aware of the distinction between auto candidate_class and human reference_class. That is an important nuance: the system does not assume the model or Otsu logic is ground truth; it uses reference labels as a policy layer when available.

• The final benchmark uses the model's parsed answer against candidate_class, not a fully independent annotation stream, which is a valid benchmark design for the repo but not a fully independent scientific validation of forest-change labels.

• The explainability layer is mostly built as a text-generation layer over evidence cards and spectral summaries. It is evidence-grounded, but it is not equivalent to an audited causal or post-hoc explainability protocol.

Chronology and evidence timeline

Stage	Likely chronology	What is actually present
Design and config	Initial ROI, dates, patch size, VLM image size	configs/pilot_kanha.yaml
Remote-sensing export	Before dataset assembly	scripts/01_export_gee.py and src/gee_utils.py

DIP / Otsu detection	Core processing stage	src/dip_utils.py, extract_bounding_boxes(), build_disturbance_triplets()
Dataset creation	Evidence-card generation	scripts/03_build_dataset.py and notebook 03
Freeze + split writing	Benchmark dataset finalization	scripts/05_freeze_jsonl_and_splits.py
Zero-shot evaluation	Benchmark stage	results/vlm_combo_comparison.json; notebooks 05–08
Fine-tuning	Final improvement stage	src/vlm_finetune.py; results/vlm_combo_qwen_vl_finetuned_ecoexplain.json

Benchmark results summary

Rank	Model family	Status	Accuracy
1	Qwen2-VL	Zero-shot	56.4%
2	PaliGemma	Zero-shot	36.4%
3	LLaVA-1.5	Zero-shot	32.7%
4	BLIP-2	Zero-shot	32.7%
—	Qwen2-VL fine-tuned	LoRA fine-tune	67.3%

Conclusion

The project is real, substantial, and codified. It is not merely an unimplemented plan. It contains a functioning remote-sensing data pipeline, multimodal evidence cards, benchmark code, and a successful Qwen fine-tune. However, the README and presentation materials overstate the certainty of some claims, particularly the degree to which the benchmark is a clean scientific validation rather than a structured evidence-engineering and model-comparison workflow.

The most defensible verdict is that EcoExplain-VLM is a legitimate pilot-scale research artifact: implemented, benchmarked, and partly validated, but still best described as a strong engineering + benchmarking prototype rather than a fully closed causal or XAI proof of concept.

Primary evidence files

• README.md

• scripts/01_export_gee.py

• scripts/03_build_dataset.py

• scripts/05_freeze_jsonl_and_splits.py

• src/dip_utils.py

• src/gee_utils.py

• src/vlm_eval.py

• src/vlm_finetune.py

• results/vlm_combo_comparison.json

• results/vlm_combo_qwen_vl_finetuned_ecoexplain.json

Prepared in the workspace using the repository’s actual artifacts and verified benchmark outputs.